

Improving Signal-Loss Decision in Pocket SDR

Date: 2026-06-15

1. Summary

Pocket SDR currently declares loss of a tracked signal using a single criterion: the estimated carrier-to-noise density ratio C/N0 falling below a threshold. Lowering that threshold to keep weak signals causes a failure mode in which a channel that has clearly lost the signal is *not* declared lost, because C/N0 measures only received power and is blind to the loss of carrier phase / code coherence.

This document surveys the loss-of-lock detection methods used in general GNSS receivers, proposes additions that fit the existing Pocket SDR tracking loop, explains how the associated thresholds should be determined on a principled (statistical) basis rather than by trial and error, and lists implementation notes.

2. Current Problem

The loss test is the single statement in `track_sig()` ([src/sdr_ch.c:726-737](#)):

```
double t_cn0 = !strcmp(ch->sig, "L6", 2) ? THRES_CN0_L6 : sdr_thres_cn0_u;
if (ch->cn0 < t_cn0) {
    ch->state = SDR_STATE_IDLE;
    ...
}
```

`ch->cn0` is produced by `CN0()` ([src/sdr_ch.c:570-589](#)):

```
cn0 = 10*log10( sum|P|^2 / sum|N|^2 / T )
```

where **P** is the prompt correlator and **N** is a fixed-offset correlator used as a noise reference (`POS_CORR_N = -120` samples). The result is smoothed with `FILT_CN0 = 0.5`.

Two root causes make this miss real losses:

1. $|P|^2 = I_P^2 + Q_P^2$ is **blind to carrier phase lock**. When the PLL loses phase lock and the carrier spins, I and Q rotate but the total prompt power is unchanged, so C/N0 stays high and no loss is declared. Lowering the threshold widens this blind zone. This is exactly the reported symptom.

2. **The noise reference is a single fixed-offset correlator.** When the loop latches onto a cross-correlation peak, a code sidelobe, or noise, the prompt power can still exceed the reference, so the ratio never drops far enough.

In short, C/N_0 monitors *presence of power* but not *coherence* (the health of carrier phase, frequency and code tracking). General receivers monitor both with separate detectors.

3. Existing Algorithms and Proposed Methods

3.1 Loss-of-lock detectors used in general receivers

Type	Indicator	What it measures	Blind spot it covers
C/ N_0 monitoring (current)	power ratio (e.g. sumP/sumN , or NBP/WBP)	received power	none for phase lock
Carrier lock detector (PLI)	$\cos 2\phi_{\text{bar}} = ((\text{sum } I)^2 - (\text{sum } Q)^2) / ((\text{sum } I)^2 + (\text{sum } Q)^2)$ (Van Dierendonck), or $(I^2 - Q^2) / (I^2 + Q^2)$	carrier phase coherence; ->+1 locked, ->0 spinning; amplitude independent	the main C/ N_0 blind spot
PLL jitter / phase-error variance	RMS of discriminator err_phas	tracking stability	loop divergence
FLL lock detector	normalized frequency residual during pull-in	pull-in / false lock	initial-acquisition false lock
Code lock detector (DLL)	$P > E, L$ and E/L symmetry	correlation-peak shape	cross-correlation / noise tracking
Bit / frame sync, parity / CRC	decode success	true confirmation of lock	final check of all blind spots
Lock counters with hysteresis	optimistic / pessimistic counters (Kaplan)	debounced decision over K epochs	over-sensitive vs. over-slow
Signal Quality Monitoring (SQM)	multi-correlator distribution	distortion / multipath / spoofing	abnormal tracking

The textbook arrangement (Kaplan & Hegarty ch. 5; Van Dierendonck; Borre SoftGNSS) is: declare loss when **C/ N_0 (power) OR carrier lock detector (coherence)** indicates loss, debounced by a pessimistic counter.

3.2 Proposed additions for Pocket SDR

Pocket SDR already computes the quantities required, so the change is low cost.

Proposal A (primary): carrier lock detector ($\cos 2\phi$).

A correct carrier lock indicator must collapse toward 0 when the carrier spins. The data-wipe-off accumulator already in `DLL()`, `sumI[i] += C[i][0] * sign(IP)` (`src/sdr_ch.c:517-520`), is *not* suitable: because `sign(IP)*IP = |IP|`, `aveI[0]` is the mean of `|I_P|`, which stays large even when the phase rotates (its no-coherence value is about $(2/\pi)^2 \approx 0.4$, not the noise floor). Reusing it as a PLI would fail to detect exactly the case we care about.

Instead use the squaring (Van Dierendonck) phase lock indicator, which removes data/secondary-code polarity without any wipe-off and goes to 0 when unlocked:

$$PLI = \frac{\text{sum}(I_P^2 - Q_P^2)}{\text{sum}(I_P^2 + Q_P^2)} \quad \text{in } [-1, 1], \quad \approx \cos(2*\phi_{\text{bar}})$$

This needs one new accumulator. In `CN0()` the denominator already exists as `sumP = sum(I_P^2 + Q_P^2)` (`src/sdr_ch.c:572-573`); add `sumD = sum(I_P^2 - Q_P^2)` over the same `T_CN0` window and compute `PLI = sumD / sumP` next to `cn0`. PLI → +1 under phase lock (Costas or 4-quadrant; the squaring is polarity independent) and → 0 when the carrier spins or there is only noise, so it catches the "high power, no phase lock" case that C/N0 misses.

Proposal B: replace the single instantaneous test with a pessimistic counter and an OR condition.

Critically, `track_sig()` runs every code epoch (1 ms for L1 C/A), but `cn0` and `PLI` are only refreshed once per `T_CN0` window. The loss decision and the counter must therefore be evaluated **only at the `T_CN0` boundary, when `CN0()` produces a fresh decision value** - not every epoch. Otherwise `K` counts code epochs and `K = 2-4` becomes a 2-4 ms debounce instead of the intended 1-2 s.

```
// run only when CN0() has just updated cn0/PLI (T_CN0 boundary):
bad = (cn0 < t_cn0) || (ch->costas && pli_valid && PLI < t_pli); // power OR coherence
if (bad) lost_cnt++; else lost_cnt = 0;
if (lost_cnt >= K) -> SIGNAL LOST;           // K counts T_CN0 windows, not epochs
```

The OR captures "power present, phase lost"; the counter prevents false losses from brief fades, so the threshold can be lowered safely. `pli_valid` gates the coherence term until the first `PLI` is available (see implementation notes).

Proposal C (deferred): fold sync-quality signals into the decision later. Secondary-code loss

(`THRES_LOST`, `src/sdr_ch.c:462`) currently only resets `sec_sync`; for signals with a secondary code a sustained collapse is a strong loss indicator. Note that `ch->nav->fsync` is *not* a continuous quality metric - it stores the lock count at which frame sync was established and is reset on loss (`src/sdr_ch.c:731-732`), so it does not by itself signal "frame sync broke during tracking." Using the nav layer for loss detection requires a dedicated counter (consecutive CRC/parity failures, symbol-sync lost, or nav-decode error rate). Keep these as log-and-verify only at first and add them to the loss condition after their real-data distributions are examined.

Proposal D (optional): code lock detector. Use `aveP[0]` (P) vs `aveP[1]/aveP[2]` (E/L) to require `P > max(E, L)` with $E \sim L$, rejecting noise / sidelobe tracking (flat $P \sim E \sim L$ correlation).

Recommended first step: **A + B** (the $\cos^2\phi$ carrier lock detector plus a pessimistic OR decision). This is the minimal change that addresses the root cause (coherence is unmonitored); it needs one new accumulator (`sumD`) rather than zero, but no new correlators. Proposals C and D are deferred: keep the secondary-code / symbol / nav-decode signals as log-and-verify only at first (section 6) and fold them into the loss condition only after their real-data distributions are examined.

4. Threshold Determination Criteria

Thresholds are not tuned arbitrarily; each is derived from the statistical distribution of its metric under the locked (H1) and lost (H0) hypotheses, trading false-alarm probability P_{fa} (declaring loss while still locked) against miss probability P_{md} .

Common principle (binary hypothesis test per decision window). For a metric where *lower means worse* (both C/N0 and PLI), with H1 = still locked and H0 = lost / noise:

- The lower tail of H1 (locked at the design-minimum C/N0) that falls below the threshold sets the false-loss probability P_{fa} (declaring loss while still locked). Raising the threshold increases P_{fa} , so P_{fa} gives the **upper** bound on the threshold.
- The upper tail of H0 (lost / noise) that stays above the threshold sets the missed-loss probability P_{md} (not declaring loss while actually lost). Lowering the threshold increases P_{md} , so P_{md} gives the **lower** bound on the threshold.
- Place the threshold between the two distributions. If they overlap, **increase the averaging length M to separate them first**, then place it.
- Anchor to the loop tracking threshold and add hysteresis (lock > lost). This is why the current code uses the 34/30 dB-Hz pair.

4.1 C/N0 threshold (currently 30/34, L6 = 33)

Basis: the carrier-loop tracking threshold. Costas-loop thermal-noise phase jitter (Kaplan):

$$\text{sigma_phi} = (180/\pi) * \text{sqrt}(B_n / (C/N0) * [1 + 1/(2*T*(C/N0))]) \quad [\text{deg}]$$

with `Bn` the loop noise bandwidth (`B_PLL` = 5 Hz) and `T` the coherent integration time (`ch->T`). The tracking-threshold rule is `3*sigma_phi <= pull-out (+45 deg for a Costas loop)`, i.e. thermal-only `sigma_phi <= 15 deg`. Solving for C/N0 gives the physical lower bound of the loss threshold.

- This is why the value is signal dependent (it scales with `T`, `Bn`, and modulation), consistent with **L6 (CSK) using a separate 33 dB-Hz value**.
- `THRES_CN0_L = 34` is the lower bound plus about 4 dB of hysteresis margin so lock is not declared right at the edge.

- The compile-time constants are 34/30 dB-Hz, but the shipped runtime config (`app/pocket_trk/pocket_trk_default.conf`) overrides them to `thres_cn0_l = 35.0` , `thres_cn0_u = 30.5` , with `b_pll = 10.0` (and `b_d11 = 0.5` , `b_f11_w = 10.0` , `b_f11_n = 5.0`). Use those numbers when reasoning about normal operation - a larger `B_PLL` raises the C/N0 tracking threshold via the jitter formula above.

4.2 Carrier lock detector ($\cos 2\phi$) threshold (Proposal A)

Evaluate PLI over the `T_CN0` window, $M = T_CN0/ch \rightarrow T$ epochs (default $0.5/0.001 = 500$). Per-epoch model $I = A \cos(\phi) + n$, $Q = A \sin(\phi) + n$ (variance σ^2 each); squaring cancels data/secondary polarity, and the noise contributions to I^2 and Q^2 cancel in expectation:

- Locked ($\phi \approx 0$): $E[I^2 - Q^2] = A^2$, $E[I^2 + Q^2] = A^2 + 2\sigma^2$, so $PLI1 \approx A^2/(A^2 + 2\sigma^2) = 1/(1 + 1/\rho)$, with $\rho = A^2/(2\sigma^2) \approx (C/N0)*T$ the post-epoch SNR.
- Spinning / noise ($H0$): $E[I^2 - Q^2] = 0$, so $PLI0 \approx 0$, with a spread of order $1/\sqrt{M}$ about zero.

Numerical feel for `PLI1` (independent of M):

C/N0	$\rho = (C/N0)*T$ (T = 1 ms)	PLI1
40 dB-Hz	10	0.91
35	3.16	0.76
30 (tracking threshold)	1.0	0.50
25	0.32	0.24

Because $H0$ now sits near 0 (not ~ 0.4 as a mean-`|I|` metric would), the locked and lost distributions are well separated for strong signals. Place the threshold a few $H0$ standard deviations above 0 and below `PLI1` at the design-minimum C/N0; for a 30 dB-Hz design point (`PLI1  $\approx$  0.5`), **0.2-0.3** bounds both Pfa and Pmd. Near the tracking threshold `PLI1` shrinks toward the $H0$ spread; the remedy is to **increase M** (the metric is already on the `T_CN0 = 0.5 s` window; extend it further if needed), which tightens the $H0$ spread as $1/\sqrt{M}$ while leaving `PLI1` unchanged - separate the distributions before adjusting the threshold.

4.3 Lost counter K (Proposal B)

K is fixed by the intersection of two requirements:

- False-loss suppression: with single-test Pfa = p, K consecutive bad epochs have probability $\sim p^K$; from an acceptable false-loss rate, $K \geq \log(\text{acceptable})/\log(p)$.
- Reaction latency: $K * \text{decision_period} \leq$ acceptable loss-detection delay.

Take the smallest K satisfying both. With a `T_CN0 = 0.5 s` decision period, K = 2-4 gives a 1-2 s debounce.

4.4 Code lock threshold (Proposal D)

From the E/L noise statistics: require $(P - \max(E, L)) / \sqrt{\text{noise power}} > k \cdot \sigma$ with k (e.g. 3) chosen from the target Pfa - the same noise-floor basis as C/N0.

4.5 Final calibration on real data

After fixing the averaging windows analytically, confirm the numeric values from real data; Pocket SDR makes this straightforward:

1. Receive a live signal, then force a loss (disconnect antenna / fade / attenuate).
2. Log `cn0` and the new `PLI` (via `$LOG` or `sdr_ch_corr_hist`).
3. Build histograms of each metric over known-locked and known-lost segments; place the threshold at the crossover, or per the Pfa/Pmd targets (Pfa from the H1 / locked lower tail, Pmd from the H0 / lost upper tail; see section 4).
4. Tabulate per signal type (different `ch->T`, `B_PLL`, modulation), as already done for L6.

5. Implementation Notes

- **One new accumulator, no new correlators.** Do not reuse `aveI[0]`: it is the mean of `|I_P|` (the `sign()` is the sign of `I_P` itself, `src/sdr_ch.c:517-520`), not a coherent in-phase sum, so it does not collapse when the carrier spins. For the cos-2*phi PLI add `sumD = sum(I_P^2 - Q_P^2)` next to the existing `sumP` in `CN0()` (`src/sdr_ch.c:572-573`), accumulated from the same prompt history `P[SDR_N_HIST-1]`, and divide at the `T_CN0` boundary.
- **Storage.** Add `sumD` to `sdr_trk_t` and a `pli` (and a `lost_cnt`) to the channel structs in `src/pocket_sdr.h` (`sdr_ch_t` around `pocket_sdr.h:223-226` / `sdr_trk_t`).
- **PLI initial value / valid flag.** `PLI` only becomes meaningful after the first `T_CN0` window. `start_track()` initializes `ch->cn0` with the acquisition C/N0, but `pli` is new, so define its startup behavior explicitly: carry a `pli_valid` flag (set on the first `CN0()` update) and gate the coherence term on it, or initialize `pli = 1.0` (fully locked). Either way the `PLI < t_pli` term must not fire before the first decision value exists, or a freshly acquired channel would be declared lost immediately. This is the natural counterpart to evaluating the loss decision only on `CN0()` updates.
- **API exposure (compatibility).** If `pli` is surfaced through `sdr_ch_corr_stat()`, the `stat[]` array order is effectively public; only **append** the new field at the end and check the expected length on the Python / GUI / `test/` side before relying on it. Likewise, adding `pli` to a `$CH` log column affects existing CSV consumers - treat that as a separate, explicit compatibility decision rather than a silent change.
- **Make thresholds tunable globals**, mirroring `sdr_thres_cn0_l/u` (`src/sdr_ch.c:65-66`): add `sdr_thres_pli`, `sdr_lost_cnt` (K). This keeps calibration in section 4.5 a runtime/config change rather than a recompile, and matches the existing options style (System / Signal Options).
- **Per-signal-type values.** `PLI1` depends on `ch->T` and the post-epoch SNR, and the H0 spread depends on `M = T_CN0/ch->T`; keep separate defaults where modulation differs (as L6 already does for C/N0).

- **Applicability via `ch->costas` , not signal name.** Gate the PLI term on `ch->costas` (the normal prompt-tracking case) rather than excluding L6D/E by name. The `cos 2*phi` indicator assumes a Costas-tracked prompt; CSK / non- Costas paths (currently L6D/E, where `ch->costas` is false) stay on the C/N0 path. Keying on the mode flag is robust to future signal additions.
- **Window.** Evaluate PLI directly on the `T_CN0` window (the same cadence as the C/N0 update), giving $M = T_CN0 / ch->T$ epochs for the separation in section 4.2. Reset `sumD` together with `sumP` / `sumN` at each C/N0 update.
- **Hysteresis and counter reset.** Reset `lost_cnt` to 0 on any good decision window (not every epoch); declare loss only at `lost_cnt >= K`. Reset `lost_cnt` (and `pli_valid`) together with the other per-lock state already cleared on loss / re-acquisition.
- **Keep it a root-cause change, not compensation.** The added metric measures coherence directly; do not introduce power-side fudge factors to mask the blind spot.
- **No NULL checks needed** around `sdr_malloc` if any allocation is added; it aborts on failure by project convention.

6. Other

- **Scope.** This change concerns only the loss decision in the tracking state. Acquisition (`THRES_CN0_L` gating in `srch_sig()` , [src/sdr_ch.c:419-432](#)) is unaffected, though the same PLI could later be used to confirm a clean pull-in before declaring lock.
- **Interaction with bump-jump.** Bump-jump ([src/sdr_ch.c:543-567](#)) already corrects BOC false locks; the code lock detector (Proposal D) is complementary and should not duplicate it.
- **Validation.** Beyond the histogram calibration, validate against ground-truth lock via parity/CRC decode success per signal, and check that weak-but-valid signals near the design-minimum C/N0 are retained (low Pmd) on recorded IF data.
- **References.** Kaplan & Hegarty, "Understanding GPS/GNSS", ch. 5 (tracking-loop thresholds, lock detectors); Van Dierendonck (carrier lock detector, NBP/WBP); Borre et al., "A Software-Defined GPS and Galileo Receiver" (SoftGNSS lock logic).